

FINAL EXAM MOCK

AWS HANDS-ON PRACTICE & NETWORK SECURITY



DESIGN & DEPLOY HIGHLY AVAILABLE WEB ARCHITECTURE ON AWS

Created by **Hussain Ali**
and Moderated by **Ali AlRashedi**

Hussain Ali
NETWORKING

Final Exam Mock

This mock exam is designed to help you practice and apply everything you've learned throughout the AWS course. It includes hands-on tasks that cover core topics such as VPC configuration, EC2 deployment, load balancing, storage, networking, and security.

Disclaimer:

This mock is for **training purposes only**. These exact questions are **very unlikely** to appear in the final exam.

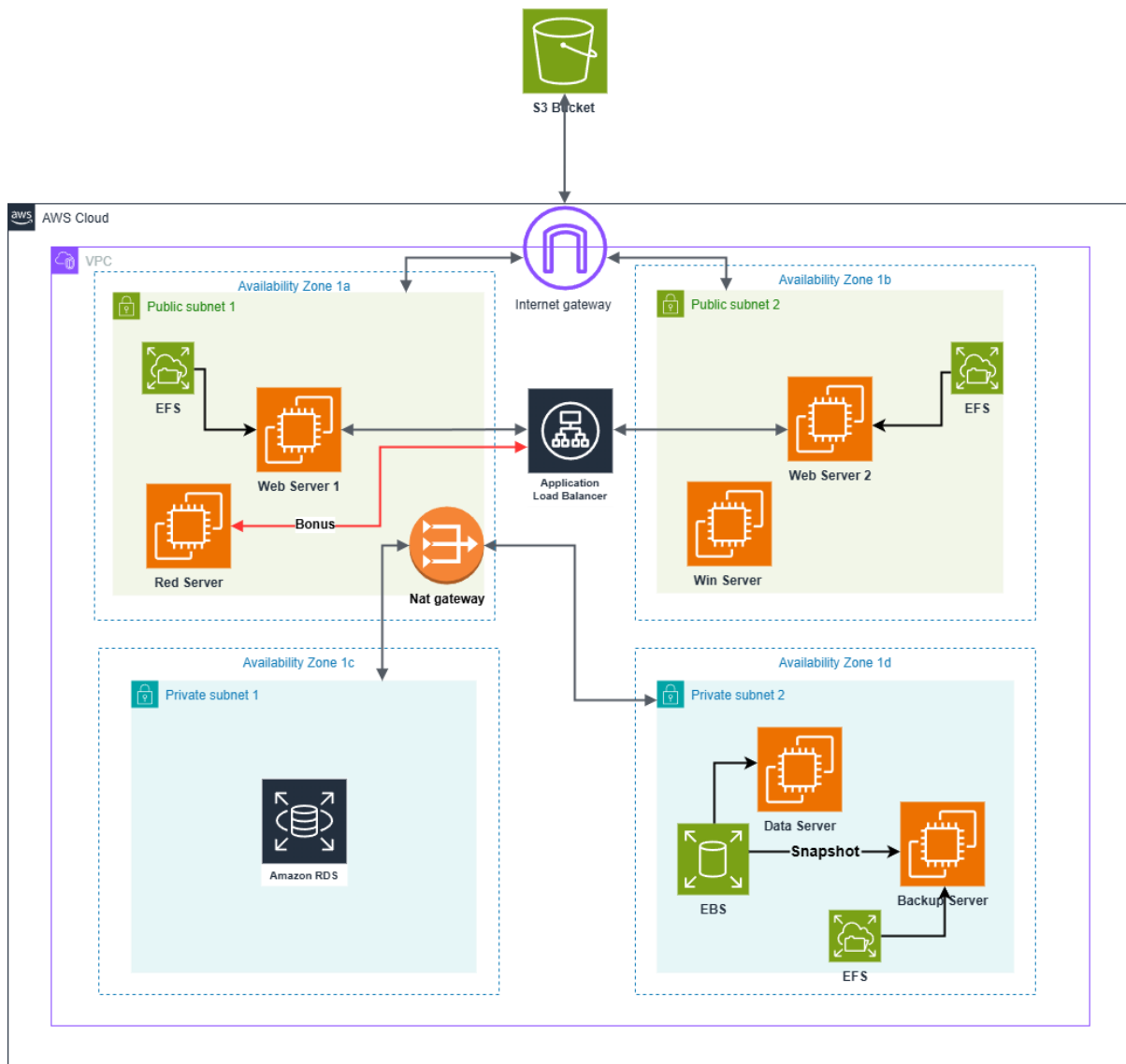
However, if you complete this mock properly, it will test you on **every major concept** required for the exam and **significantly improve** your chances of passing.

Take it seriously. Follow each task. Document your work with screenshots as instructed.

The Mock may take **1 hour and 30 minutes** to complete if all the steps are followed correctly (you may be able to complete it earlier if you multitask).

Note: You can find the answer key in the [YouTube Channel](#).

This document is open source, created by Hussain Ali and moderated by Ali AlRashedi.



You have been asked to design and deploy a highly available web architecture using AWS. The solution must span **four Availability Zones (AZs)** and include the following requirements:



Two public subnets in different AZs, hosting **one Ubuntu EC2 instance** acting as a **Web Server** in each public subnet, **one Windows Server**, and **one Red Hat Server**.



Two private subnets, in different AZs, hosting backend services like **RDS**, a **data server**, and a **backup server**.



A **NAT Gateway** must be deployed in one of the public subnets to allow outbound internet access from instances in private subnets.



A single **Application Load Balancer (ALB)** must route traffic to web servers across AZs.



Use **EFS and EBS** as shown for storage solutions.

Note: All EC2 configuration and validation must only be performed via the command line CMD. You are required to connect to your EC2 instances using SSH, provide screenshots of all outputs from the terminal, and use Remote Desktop Protocol (RDP) for the Windows Server.

Do not use the AWS Console to interact directly with the EC2 instances.

Task 1: VPC and Subnet Configuration

-  Create a **VPC** named “**MockVPC**” with a **Class C, CIDR block**.
-  Create **two public subnets** and **two private subnets** across **4 Availability Zones**.
-  Attach an **Internet Gateway** Called “**Mockig**” to the VPC.
-  Create a **NAT Gateway** called “**Mock-NAT**” in the public subnet to connect the private subnet.

Task 2: EC2 and Load Balancer Configuration

-  Launch **one Ubuntu EC2 instance** in each **public subnet**.
-  Install a **Web Server** on each **public subnet**.
-  Install **Windows Server 2022 Base** on **Win Server**.
-  Install **Red Hat Linux** on **Red Server**.
-  Launch an **Ubuntu EC2 instance** for the **Data server** and **Backup Server** in **private subnet 2**.
-  Create **security groups (SG)**:

1. **Public subnet SG** should allow **SSH, HTTP/HTTPS, ICMP, MYSQL/Aurora, RDS,** and **NFS**.
 2. **Private subnet SG** should only allow access from the public subnet SG (web servers) and **NFS**.
-  Create an **Application Load Balancer** to balance traffic between the two Web Servers and demonstrate its working.
-  Register the Web Servers in the ALB's target group and verify the setup.

Task 3: Backend and Storage Configuration

1. In private subnet 1:
 -  Deploy **Amazon RDS** or **EC2-based DB** instance.
 -  Set the master username to **admin** and the password to **master100**.
 -  Configure availability zone to **no preference**.
2. In private subnet 2:
 -  In the **Data server**:
 -  Use **EBS** volumes for storage (30 GiB **encrypted**).
 -  Create a text file called **EBSmock.txt** inside the volume.
 -  In **Backup server**:
 -  Create a **snapshot** from the EBS volume and mount it on the **Backup server** and show the **EBSmock.txt**.
3. Configure and verify shared file access using **Amazon EFS**
 -  Create an **EFS file system** with mount targets in the required subnets.
 - Mount the **EFS file system** on:
 - **Both web servers** in the **public subnets**.
 - The **Backup server** in **private subnet 2**
 -  In **one instance**, create a text file named **EFSmock.txt** within the mounted EFS directory.
 -  Verify that the **EFSmock.txt** file is accessible from all instances connected to the **EFS file system**.

Task 4: S3 Bucket Configuration and Integration

1. Create an S3 Bucket

-  Enable **Access Control List (ACL)** for granular permission control.
-  Create an **index.html** file that includes **your name**.
-  Upload index.html to the S3 bucket and configure **Static website hosting**.

Task 5: Validation and Screenshots

✓ In the **Screenshots, Show that:**

-  Web servers can be accessed via the load balancer's DNS.
-  Instances in private subnets can access the internet through the NAT Gateway.
-  RDS is accessible from the EC2s in the public subnet.
-  EBS volume is attached and mounted correctly.
-  EFS is shared and accessible between public EC2s and the private subnet 2.
-  Public EC2 Instances should be able to ping each other.
-  Can access the static web page from the internet.
-  Can access the Ubuntu and Red Hat Servers from the CMD and Windows Server from the RDP.

Task 6: Bonus Tasks

-  Create an **Application Load Balancer** to balance traffic between the **Red Hat server** and **web servers 2** and demonstrate its working.
(2 web servers + Red Hat in 1 load balancer)

Tip: Use NGINX instead of httpd on Red Hat Linux

-  Connect to **Windows Server using SSH**.

Note: This task is for practice only. It **is very unlikely that similar** questions will appear in the actual exam.

GOOD LUCK :)